Digital to analog conversion:

Digital-to-analog conversion is the process of changing one of the characteristics of an analog signal based on the information in digital data.

Three meachanisms for modulating digital data into an analog signal:

(i) Amplitude Shift Keying(ASK)

(ii) Frequency Shift Keying(FSK)

(iii) Phase Shift Keying(PSK)

```
import numpy as np
import matplotlib.pyplot as plt
import math
```

```
def compute_Unipolar_NRZ_Encoding(bits, N, bit_rate, T, Time):
  x = np.array([])
  for i in range(N):
    temp = bits[i]* np.ones((T))
    x = np.append(x, temp)
  t = np.linspace(0,N*Time,len(x))
  return [t, x]
```

```
def plot_signal(t, x):
  plt.figure(figsize=(15, 5))
  plt.plot(t, x)
  plt.show()
```

(i) Amplitude shift keying(ASK)

The amplitude of the carrier signal is varied to create signal elements, where both frequency and phase remain constant.

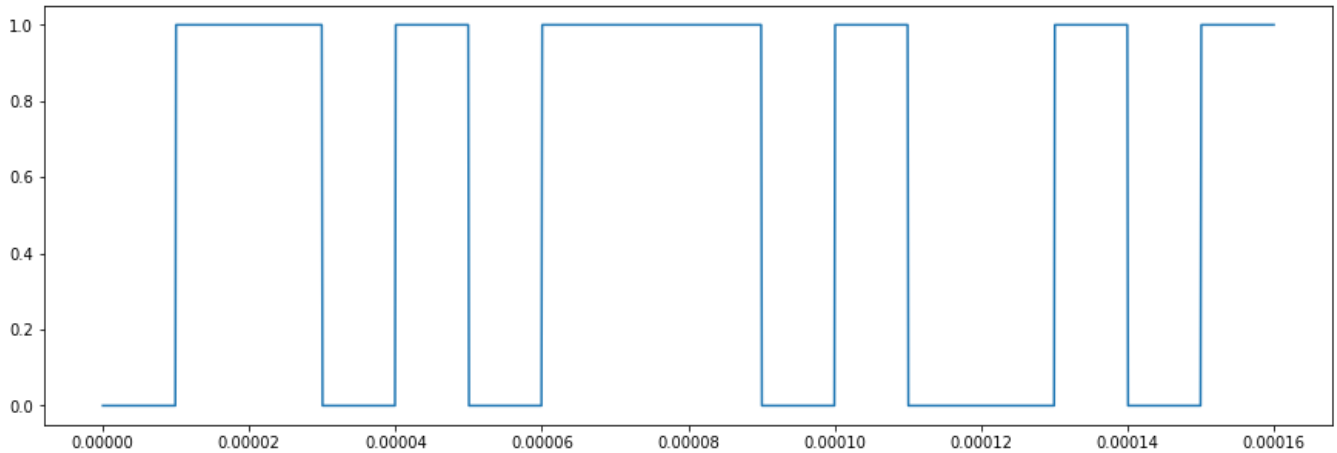
using only two level of amplitude is called BASK or binary ASK.

BASK Implementation(unipolar , non return to zero)

step 1: unipolar NRZ encoding

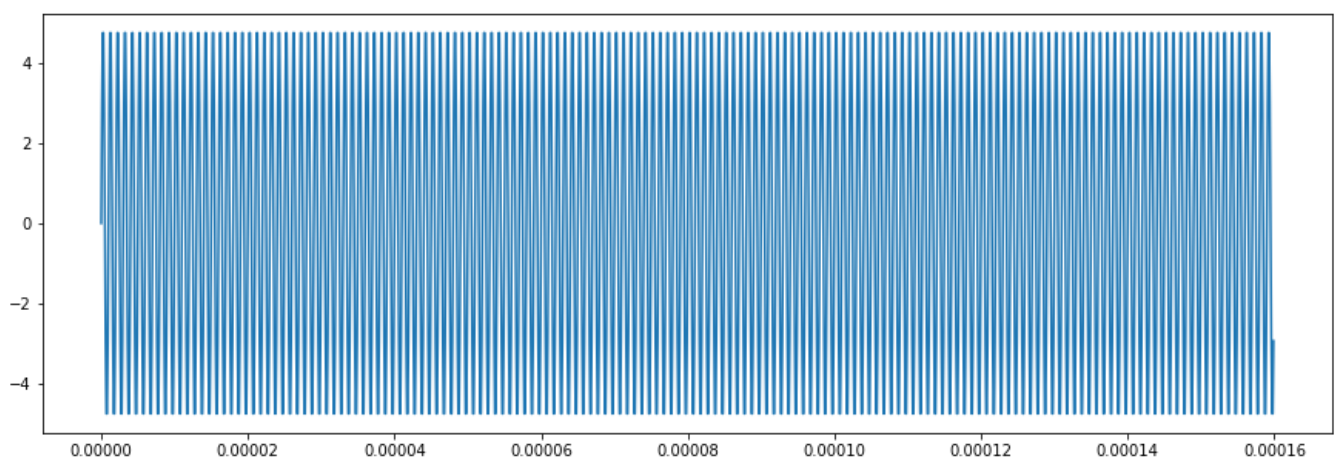
```
bits = np.array([0, 1 ,1, 0, 1, 0 ,1, 1, 1, 0, 1, 0, 0, 1, 0, 1])
Time = 10e-6
bit_rate = 1/Time
```

```
N = len(bits)
T = 100
t,x = compute_Unipolar_NRZ_Encoding(bits, N, bit_rate, T, Time)
plot_signal(t, x)
```



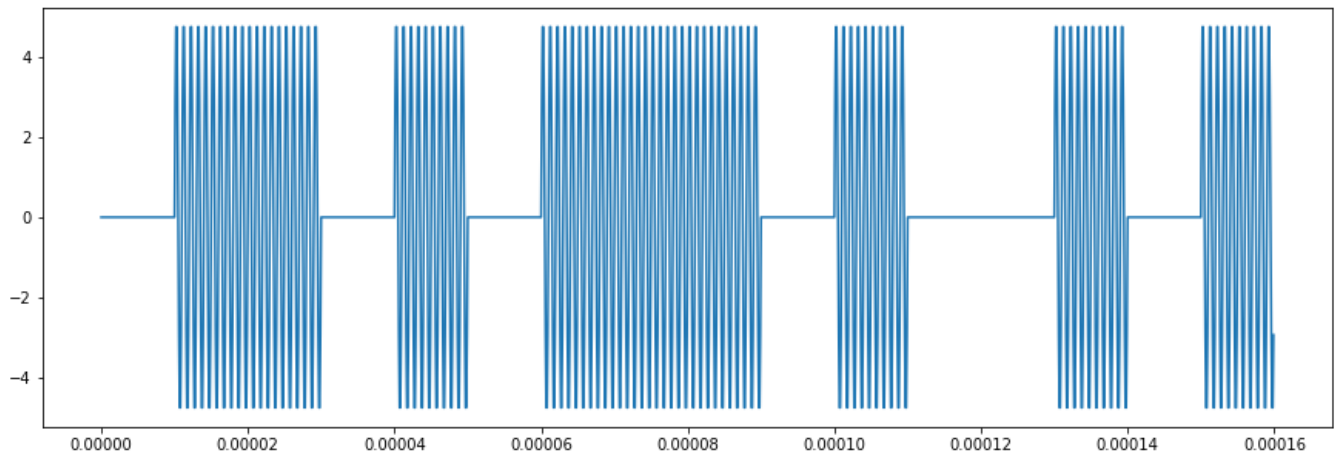
step 2: Create a Carrier signal

```
Ac = 5
# Ac - Amplitude of the carrier signal
fc =round(10*bit_rate)
# fc - Frequency of the carrier signal
tc = np.arange(0,N*Time,Time/T)
# tc - Time variable of the carrier the signal
xc = Ac * np.sin(2.0 * np.pi * fc * tc)
# xc is the Carrier signal
plot_signal(tc, xc)
```



step 3: Multiplying NRZ digital signal by the carrier signal

```
modulated_signal = x * xc
plot_signal(tc, modulated_signal)
```



(ii) Frequency Shift Keying(FSK)

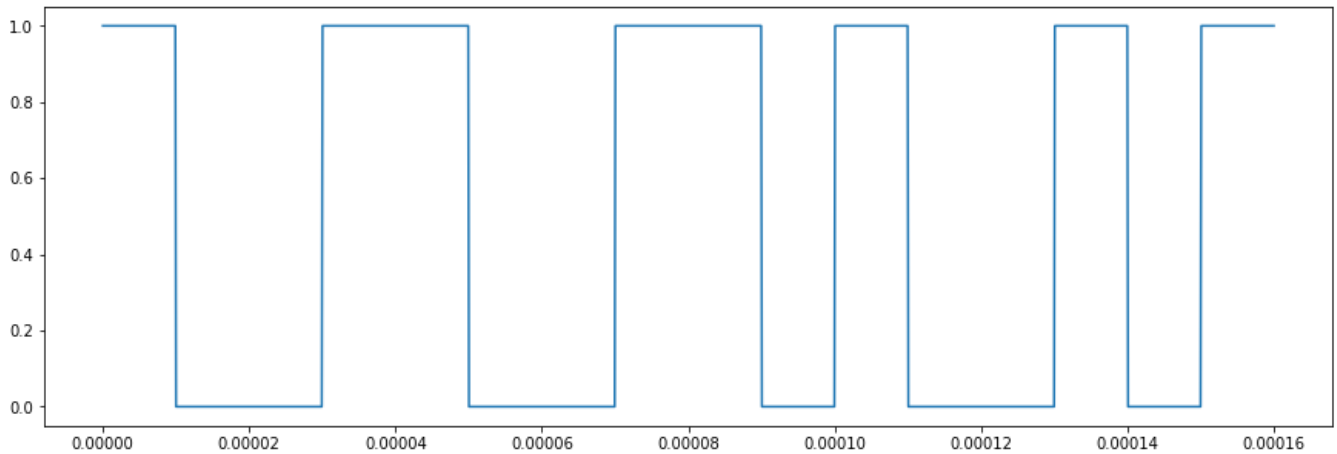
The Frequency of the carrier signal is varied to create signal elements, While both amplitude and phase remain constant.

FSK is normally implemented using two carrier frequencies. This process is known as Binary FSK or BFSK

BFSK implementation

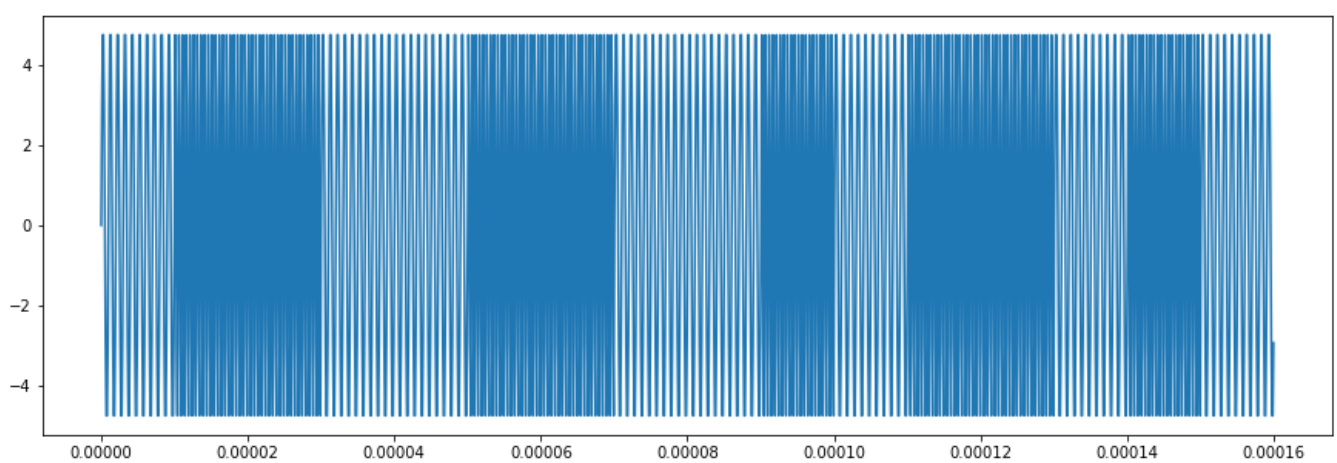
step 1: unipolar NRZ encoding

```
bits = np.array([1,0,0,1,1,0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1])
Time = 10e-6
bit_rate = 1/Time
N = len(bits)
T = 100
t,x = compute_Unipolar_NRZ_Encoding(bits, N, bit_rate, T, Time)
# x - Unipolar-NRZ encoded signal
plot_signal(t, x)
```



step 2: Assigning one carrier signal with two types of frequencies for two levels.

```
A = 5
fc1 =round(10*bit_rate)
fc2 =round(20*bit_rate)
tc = np.arange(0, Time, Time/T)
fsk_modulated_signal = np.array([])
for i in range(N):
  if bits[i] == 1:
    sig = A * np.sin(2.0 * np.pi * fc1 * tc)
    fsk_modulated_signal = np.append(fsk_modulated_signal, sig)
  else:
    sig = A * np.sin(2.0 * np.pi * fc2 * tc)
    fsk_modulated_signal = np.append(fsk_modulated_signal, sig)
plot_signal(t, fsk_modulated_signal)
```



(iii) Phase Shift Keying(PSK)

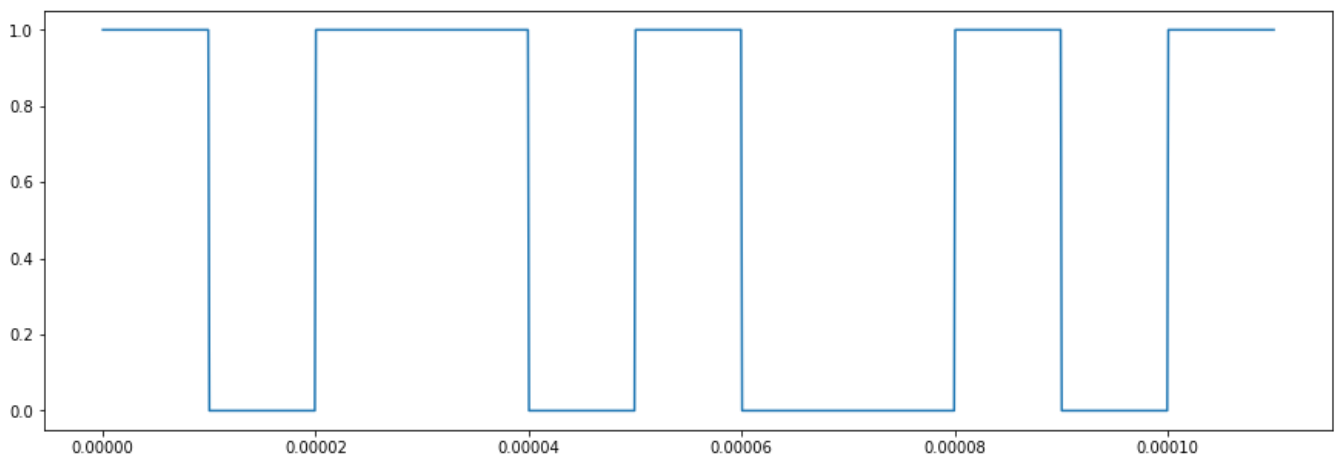
The Phase of the carrier signal is varied to create signal elements, While both amplitude and frequency remain constant.

PSK is normally implemented using only two phases. This process is known as Binary PSK or BPSK.

implementing BPSK

step 1: unipolar NRZ encoding

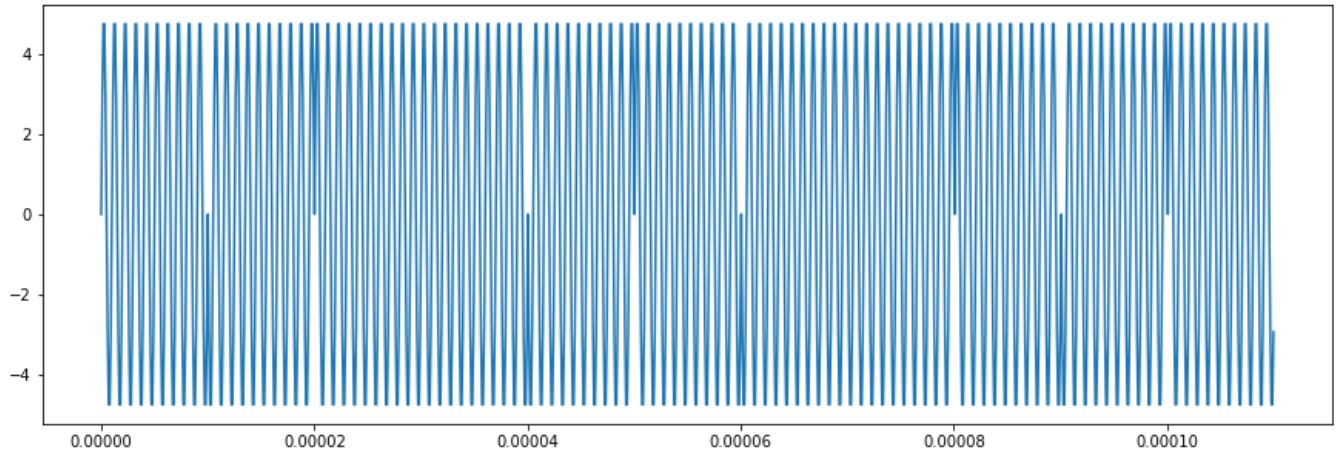
```
bits = np.array([1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1])
Time = 10e-6
bit_rate = 1/Time
N = len(bits)
T = 100
t,x = compute_Unipolar_NRZ_Encoding(bits, N, bit_rate, T, Time)
# x - Unipolar-NRZ encoded signal
plot_signal(t, x)
```



Step 2: Assigning one carrier signal with two types of phases for two levels.

```
A = 5
fc =round(10*bit_rate)
ph1 = 0
ph2 = np.pi
tc = np.arange(0, Time, Time/T)
psk_modulated_signal = np.array([])
for i in range(N):
  if bits[i] == 1:
    sig = A * np.sin((2.0 * np.pi * fc * tc) + ph1)
```

```
    psk_modulated_signal = np.append(psk_modulated_signal, sig)
  else:
    sig = A * np.sin((2.0 * np.pi * fc * tc) + ph2)
    psk_modulated_signal = np.append(psk_modulated_signal, sig)
plot_signal(t, psk_modulated_signal)
```



✓ 0s completed at 9:13 PM